

HACKATHON EDN

Analise Inteligente de Documentos de Sinistro

com Strands Agents + AWS Serverless

Duracao 1h a 1h30	Formato Times de 3 a 4 pessoas	Stack Python + boto3 + Strands
-----------------------------	--	--

Escola de Nuvem (EDN) | AWS AI Services

1. Contexto

A DocuSmart Seguros precisa agilizar a análise de sinistros. Hoje, cada processo exige que um analista abra manualmente um PDF, leia o documento, identifique o tipo (boletim de ocorrência, laudo médico, nota fiscal...) e registre as informações no sistema. Isso leva cerca de 15 minutos por documento e gera erros de digitação.

O seu time foi contratado para construir, em tempo record, um micro-pipeline serverless que automatize esse trabalho. A novidade: vocês vão usar o Strands Agents SDK da AWS para criar um agente de IA que coordena toda a lógica de análise dentro de uma Lambda.

2. O que é o Strands Agents SDK?

O Strands Agents é um framework open source da AWS lançado em 2025 para construir agentes de IA de forma simples, usando Python. Em vez de escrever toda a lógica de chamada ao LLM manualmente, você define ferramentas (functions Python) e o agente decide sozinho quais usar e em que ordem para responder ao que você pediu.

Filosofia: Model + Tools + Prompt. Mínimo de código, máximo de inteligência.

Onde roda: Local, Lambda, Fargate, container — qualquer lugar com Python.

Modelo: Amazon Bedrock (Claude, Nova, Titan...) configurado por variáveis de ambiente.

A estrutura básica de um agente Strands é assim:

```
from
    strands import Agent
from
    strands.models import BedrockModel

# 1. Defina ferramentas como funções Python decoradas
@tool
def extrair_texto_documento
(bucket: str, key: str) -> str:
    """Extraí o texto de um documento PDF usando Amazon Textract."""
    # ... chama Textract ...

# 2. Crie o agente com modelo e ferramentas
agente = Agent(
    model=BedrockModel(model_id="us.amazon.nova-pro-v1:0"),
    tools=[extrair_texto_documento, salvar_resultado],
    system_prompt="Você é um especialista em análise de sinistros..."
)

# 3. Chame o agente com linguagem natural
resultado = agente("Análise o documento no bucket X, chave Y")
```

3. O Desafio

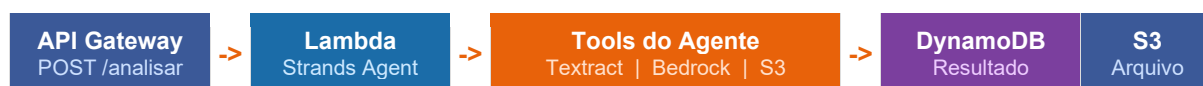
Construa uma API serverless que receba um documento de sinistro, processe-o com um agente Strands e devolva as informações extraídas de forma estruturada.

INPUT: Bucket S3 e chave (key) do arquivo enviados no corpo da requisicao POST
OUTPUT: JSON com tipo do documento, campos extraídos, resumo e ID do registro salvo

O que o agente deve ser capaz de fazer:

1. Chamar o Textract para extrair o texto do documento
2. Classificar o tipo de documento (boletim, laudo, nota fiscal...)
3. Extrair campos relevantes (data, valor, envolvidos, local...)
4. Gerar um resumo em linguagem natural
5. Salvar o resultado no DynamoDB

4. Arquitetura da Solucao



A Lambda recebe a requisicao do API Gateway, inicializa o agente Strands e passa uma instrucao em linguagem natural. O agente decide quais ferramentas usar e em que ordem — voce nao precisa codificar o fluxo passo a passo.

5. Servicos AWS Utilizados

Servico	Papel na solucao
Amazon S3	Armazena os documentos de sinistro antes de processar
Amazon Textract	Ferramenta do agente: extrai texto e tabelas do documento via OCR
Amazon Bedrock	Modelo LLM que alimenta o agente Strands (ex: Amazon Nova Pro)
Strands Agents SDK	Framework que coordena o agente e executa as ferramentas na Lambda
AWS Lambda	Hospeda o agente Strands (com Layer pre-configurado)
Amazon DynamoDB	Ferramenta do agente: salva o resultado estruturado do sinistro
Amazon API Gateway	Endpoint POST /analisar-sinistro que aciona a Lambda

6. Configurando o Strands na Lambda

Passo 1 — Adicionar o Lambda Layer do Strands

O Strands disponibiliza um Layer publico da AWS. Adicione o ARN abaixo nas camadas da sua Lambda (Python 3.12, x86_64, regioao us-east-1):

```
# ARN do Layer oficial do Strands Agents (us-east-1, Python 3.12, x86_64)
arn:aws:lambda:us-east-1:856699698935:layer:strands-agents-py3_12-x86_64:2
```

No Console AWS: Lambda > sua funcao > Layers > Add a layer > Specify an ARN

Passo 2 — Variaveis de ambiente da Lambda

```
# Configure nas variaveis de ambiente da Lambda:
AWS_REGION_NAME    = us-east-1
DYNAMO_TABLE_NAME  = sinistros-resultados
DOCUMENTS_BUCKET    = docusmart-sinistros
```

Passo 3 — Permissoes IAM da Lambda (Execution Role)

- AmazonTextractFullAccess
- AmazonDynamoDBFullAccess
- AmazonS3ReadOnlyAccess
- AmazonBedrockFullAccess

Passo 4 — Codigo base do handler (lambda_function.py)

```
import
    json, boto3, uuid
from
    strands import Agent
from
    strands.models import BedrockModel
from
    strands.tools import tool

# --- Ferramentas do agente ---
@tool
def extrair_texto_do_documento
    (bucket: str, key: str) -> str:
    """Extrai o texto de um documento usando Amazon Textract."""
    textract = boto3.client('textract')
    resp = textract.detect_document_text(
        Document={'S3Object': {'Bucket': bucket, 'Name': key}}
    )
    return ' '.join(b['Text'] for b in resp['Blocks'] if
b['BlockType']=='LINE')

@tool
def salvar_resultado_dynamodb
    (resultado: dict) -> str:
    """Salva o resultado da analise no DynamoDB e retorna o ID gerado."""
    import os
    dynamo = boto3.resource('dynamodb')
    table = dynamo.Table(os.environ['DYNAMO_TABLE_NAME'])
    resultado['id'] = str(uuid.uuid4())
    table.put_item(Item=resultado)
    return resultado['id']

# --- Handler principal ---
def lambda_handler
    (event, context):
    body = json.loads(event.get('body', '{}'))
    bucket = body['bucket']
```

```

key      = body['key']

agente = Agent(
    model=BedrockModel(model_id='us.amazon.nova-pro-v1:0'),
    tools=[extrair_texto_do_documento, salvar_resultado_dynamodb],
    system_prompt=(
        'Voce e um especialista em analise de sinistros de seguro. '
        'Use as ferramentas disponíveis para: extrair o texto do documento, '
        'classificar o tipo (Boletim de Ocorrência, Laudo Medico, Nota
Fiscal, '
        'Documento de Identidade ou Outro), extrair campos relevantes '
        '(data, valor, local, envolvidos), gerar um resumo de 2 frases '
        'e salvar o resultado. Responda APENAS com o JSON final.'
    )
)

resposta = agente(f'Analise o documento no bucket {bucket}, chave {key}')

return {
    'statusCode': 200,
    'body': str(resposta)
}

```

7. Formato de Retorno Esperado

```

{
  "id": "7f3a91bc-...",
  "tipo_documento": "Boletim de Ocorrência",
  "resumo": "Acidente de transito na Av. Paulista em 10/06/2025,
    envolvendo dois veiculos. Sem vitimas.",
  "campos_extraidos": {
    "data": "10/06/2025",
    "local": "Av. Paulista, 1000",
    "valor_prejuizo": "R$ 4.500,00",
    "envolvidos": ["Joao Silva", "Maria Souza"]
  },
  "processado_em": "2025-06-10T14:23:00Z"
}

```

8. Divisao de Tarefas Sugerida

Membro	Responsabilidade
Dev 1	Infraestrutura: criar S3, DynamoDB, Lambda (com o Layer do Strands) e API Gateway
Dev 2	Ferramenta extrair_texto_do_documento: integrar Textract e testar com um arquivo real
Dev 3	Ferramenta salvar_resultado_dynamodb + system_prompt do agente + testes end-to-end
Dev 4 (se houver)	Endpoint GET /sinistro/{id} para consultar resultados salvos + ajustes no prompt

9. Cronograma Sugerido (90 minutos)

Bloco	Tempo	O que fazer
0 - 10 min	10 min	Leitura do desafio, divisao de tarefas. Criar S3, DynamoDB e Lambda com o Layer do Strands adicionado.
10 - 30 min	20 min	Implementar a ferramenta extrair_texto_do_documento. Fazer upload de um documento de teste e validar o retorno do Textract.
30 - 55 min	25 min	Implementar a ferramenta salvar_resultado_dynamodb. Montar o agente Strands com o system_prompt e testar chamando a funcao direto.
55 - 75 min	20 min	Conectar ao API Gateway. Testar o endpoint POST end-to-end com um documento real e verificar o resultado salvo no DynamoDB.
75 - 90 min	15 min	Bonus (GET /sinistro/{id}), ajustes no prompt, preparar demo de 2 minutos.

10. Dicas Praticas

Sobre o Strands

- O agente pode chamar as ferramentas em qualquer ordem — confie no LLM para decidir
- Se o agente nao chamar a ferramenta certa, melhore o system_prompt sendo mais especifico
- Aumente o timeout da Lambda para pelo menos 60 segundos — o LLM precisa de tempo
- O Layer oficial ja inclui tudo que voce precisa: nao e necessario instalar strands-agents manualmente

Sobre os outros servicos

- Textract: o arquivo ja precisa estar no S3 antes de chamar — faca o upload primeiro
- DynamoDB: nao precisa de schema — salve o dict Python diretamente com put_item
- API Gateway: use modo Lambda Proxy — o body chega como string, use json.loads(event['body'])
- IAM: o erro mais comum e falta de permissao — verifique o Execution Role da Lambda

Documentos de teste sugeridos

- Boletim de ocorrencia (PDF texto ou imagem escaneada)
- Nota fiscal de servico (PDF ou JPG)
- Laudo medico simples (pode ser um texto colado em PDF)

Dica: o repositorio do workshop AWS de IDP tem samples prontos para usar.

11. Apresentacao Final (2 minutos)

Ao final, cada time faz uma demo rapida mostrando:

6. Uma chamada ao vivo no Postman ou curl: enviar um documento e mostrar o JSON retornado
7. O registro salvo no DynamoDB (pode mostrar o console AWS)
8. O que aprenderam sobre o Strands e o que tentariam diferente

Bora buildar! Boa sorte ao time!

Escola de Nuvem (EDN) | AWS AI Services